

Tide

Test Integrity in Developer Environments

Tide watches a lab-test PC while students use their normal tools (VS Code, CodeBlocks, terminal, Wireshark, VMware) — and even the internet. It catches AI tools, forbidden sites, old code, and USB drives, and shows the teacher what happened. It does not lock students into a special editor, and it does not grade or punish anyone automatically.

1. How it works, in short

Every student PC runs a small program called the **agent**. The teacher's laptop runs the **server** and a **console** (a website only the teacher sees).

1. The agent watches: which window is open, which website is loaded, which apps are running, whether the internet is on, USB drives, and the clipboard.
2. Known cheats (ChatGPT, Claude, Gmail, WhatsApp, a local AI app...) are caught by a simple rule list, right on the student's PC, and closed within a second.
3. Anything the rule list doesn't know is sent to the teacher's server, which asks **Jev** — an AI model made for exactly this — "is this an AI tool, or something else?" Jev answers with a confidence score.
4. If the confidence is high (90%+), Tide closes it automatically and tells the teacher. Otherwise it just shows the teacher, who decides.
5. The teacher sees one screen: every seat, colour-coded (green = fine, orange = check, red = alert), with a live feed of what happened and why.

Internet is allowed, and watched. Keeping 60 PCs offline is what fails today (students re-plug the cable or use a hotspot). So Tide lets students stay online and checks every site instead. For very strict labs there is one switch — **Internet: Blocked** — which only sends questions to offline PCs and turns the screen red if a PC goes online.

Tide never claims to be cheat-proof. It gives the teacher evidence and lets a human make the call. Every flag is timestamped and kept — nothing is deleted.

2. What Tide catches

Cheat	How it's caught
ChatGPT / Claude / Gemini / Copilot in a browser	Reads the browser's address bar; closes the tab
An AI site or app nobody listed yet	Jev looks at the window and decides

AI that runs on the PC without internet (LM Studio, Ollama)	App monitoring closes it
Mailing code to yourself, Drive, Classroom, WhatsApp Web	On the block list; closed instantly
Getting the questions early	Questions live only in Tide and are released when the teacher presses Start
Copying from a neighbour	Different students get different question sets; matching code is flagged after submission
Old saved code, or a USB drive	Tide remembers what files existed before the test started, and flags a match
Killing the agent to cheat freely	The seat turns grey within 10 seconds — itself a red flag

3. Questions people usually ask

Can't a student just close or kill the agent?

In this demo build, yes — and the seat goes grey immediately, which is itself evidence. In a real deployment the agent runs as a Windows service with admin rights, so ordinary students can't stop it.

What if a student never opens the agent at all?

Their seat never shows up as joined, so the teacher sees "59 of 60 joined" and knows exactly who to check.

Why allow the internet at all?

Because blocking it is exactly what fails today — students plug the cable back in or use a hotspot. Instead, Tide checks every site: forbidden ones close instantly, everything else is on the teacher's timeline. Strict labs can still switch to "Internet: Blocked".

If internet is allowed, can't they just search Google or Stack Overflow?

Those are flagged for the teacher as "web lookup" (not closed automatically), so the teacher decides. A lab that wants zero lookups adds them to the block list or uses "Internet: Blocked".

Why use an AI model at all — isn't a list of blocked sites enough?

New AI sites and renamed apps appear constantly. A list can't keep up. Jev looks at any unfamiliar window and classifies it, so Tide isn't stuck waiting for someone to update a list.

Why Jev instead of ChatGPT or Claude as the judge?

Jev is built to return a label and a trustworthy confidence number — not a sentence. That's what lets Tide safely decide "close this automatically" vs. "just flag it for a human." It's also much faster and cheaper, which matters when checking 60 screens at once.

Doesn't sending data to an AI model leak student information?

Only the window title and app name of unusual activity is sent — never code, names, or screenshots. And it only happens for things the simple rule list can't already recognise.

What about false alarms — a random popup stealing focus for a second?

Ordinary switching between allowed apps is never flagged. Automatic closing only happens for a high-confidence AI/messaging/remote-access match. Everything else is just shown to the teacher, who can dismiss it in one click.

Can students use VS Code, Wireshark, VMware normally?

Yes — that's the whole point. Tide watches the desktop; it doesn't replace it with a locked-down editor.

What about a student using a virtual machine (VMware)?

Tide sees the VMware window itself, but not what's inside the guest OS's own windows — that's a known, disclosed limit. For VM labs, use "Internet: Blocked" with the VM set to host-only networking, so the VM has no way out.

Can the exam clock be cheated by changing the PC's clock?

No. The teacher's server is the only clock. It tells every agent the exact time the exam ends; the student's own PC clock doesn't matter.

Does it scale to a real lab of 60 PCs?

Yes. The demo itself simulates a full 60-seat room on one laptop to prove it. The system is lightweight per seat.

What can Tide *not* catch?

A phone hidden under a desk, a friend whispering an answer, or a student determined enough to disable Windows admin protections themselves. Tide narrows down which few seats a human should walk over and check — it doesn't replace an invigilator standing in the room.

Full technical detail: [docs/ARCHITECTURE.md](#) . Full demo script and more cross-questions: [docs/DEMO.md](#) .